

How Bluesky Works 🦋

#65: Break Into Bluesky Architecture (17 Minutes)



NEO KIM

JAN 22, 2025  PAID

Get my system design playbook for FREE on newsletter signup:

✓ Subscribed

This post outlines Bluesky architecture; you will find references at the bottom of this page if you want to go deeper.

- *[Share this post](#) & I'll send you some rewards for the referrals.*

Note: I wrote this post after reading their engineering blog and documentation.

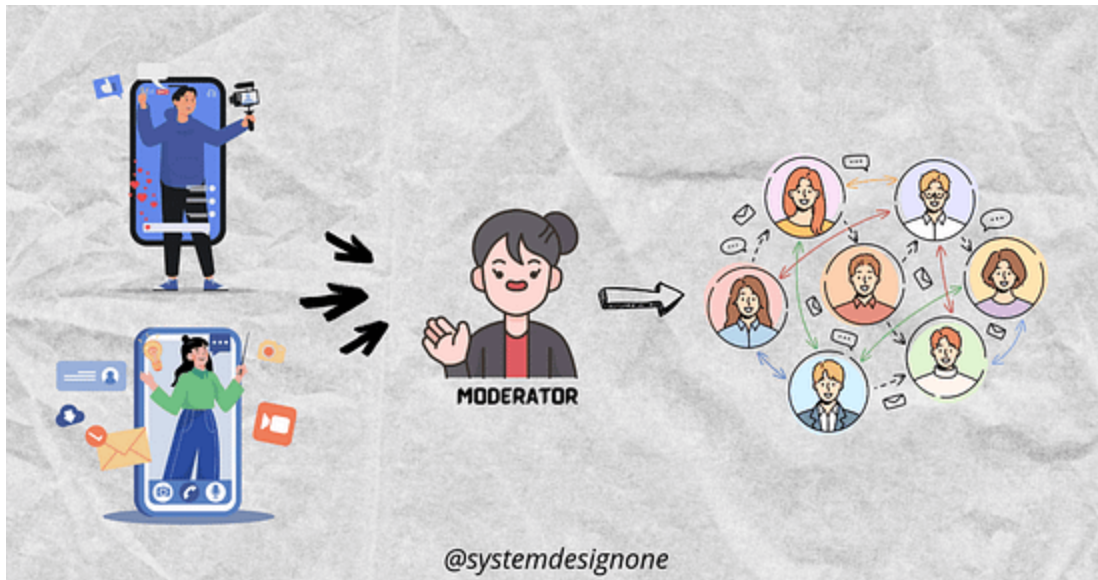
Once upon a time, Twitter had only a few million users.

And each user interaction went through their centralized servers.

So content moderation was easy.

Yet their growth rate was explosive and became one of the most visited sites in the world.

So they added automation and human reviewers to scale moderation.



Moderation in a Centralized Architecture

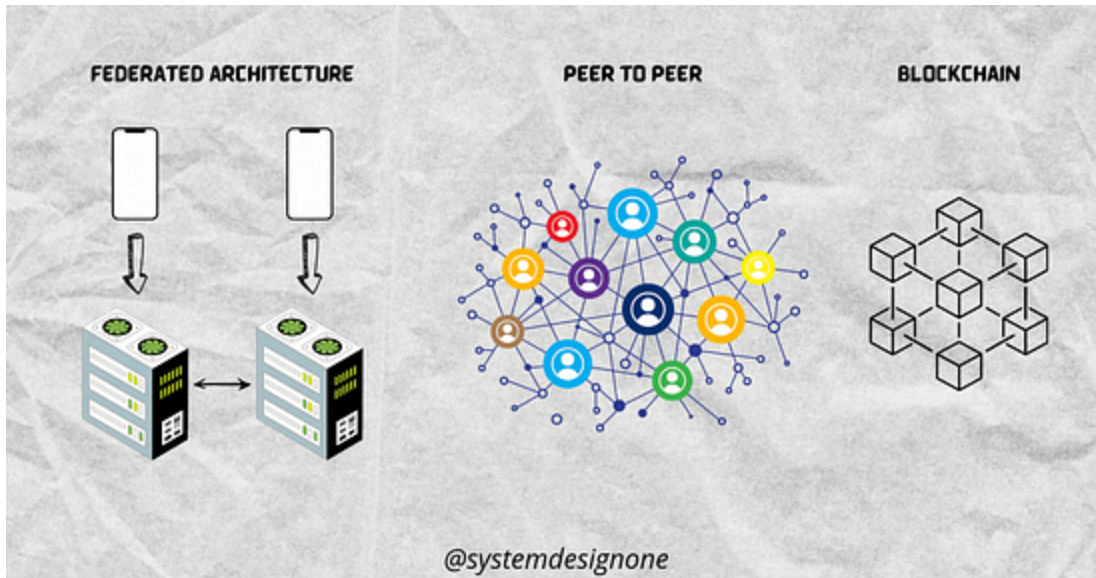
Although it temporarily solved their content moderation issues, there were newer problems. Here are some of them:

- The risk of error increases if a single authority decides the moderation policies.
- There's a risk of bias when a single authority makes moderation decisions.
- Managing moderation at scale needs a lot of effort; it becomes a bottleneck.

So they set up Bluesky: a research initiative to build a decentralized social network.

Onward.

A decentralized architecture distributes control across many servers.



Popular Decentralized Architectures

Here are 3 popular decentralized architectures in distributed systems:

- Federated architecture: client-server model; but different people run parts of the system and parts communicate with each other.
- Peer-to-Peer architecture: there's no difference between client and server; each device acts as both.
- Blockchain architecture: distributed ledger for consensus and trustless interactions between servers.

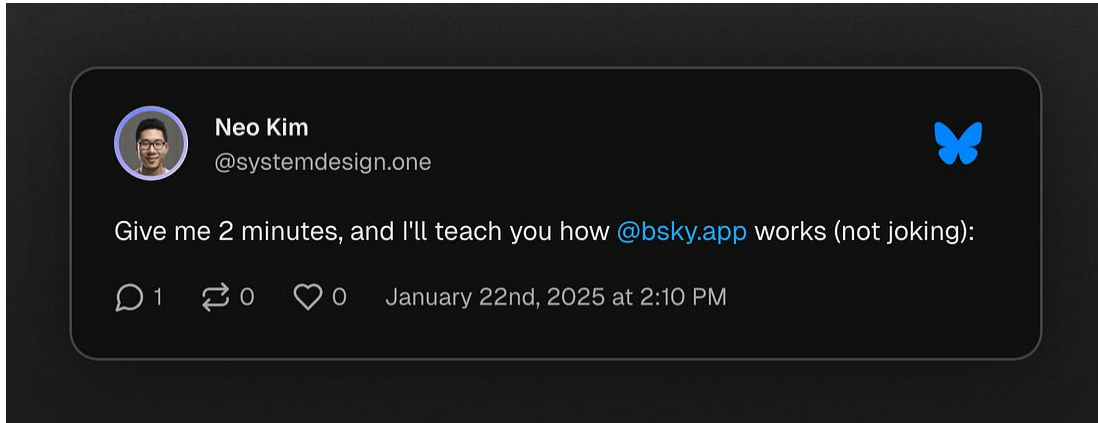
Bluesky uses a federated architecture for its familiar client-server model, reliability, and convenience. So each server could be run by different people and servers communicate with each other over HTTP.

Think of the federated network as email; a person with Gmail can communicate with someone using Protonmail.

Yet building a decentralized social network at scale is difficult.

So smart engineers at Bluesky used simple ideas to solve this hard problem.

I wrote a summary of this post (save it for later):



How Does Bluesky Work

They created a decentralized open-source framework to build social networking apps, called Authenticated Transfer Protocol (**ATProto**), and built Bluesky on top of it.

Put simply, Bluesky doesn't run separate servers; instead, ATProto servers distribute messages to each other.

A user's data is shared across apps built on ATProto; Bluesky is one of the apps. So if a user switches between apps on ATProto, such as a photo-sharing app or blogging app, they don't lose their followers (social graph).



Bluesky Explained With an Analogy

Imagine ATProto as Disneyland Park and Bluesky as one of its attractions. A single ticket is enough to visit all the attractions in the park. And if you don't like one of the attractions, try another one in the park.

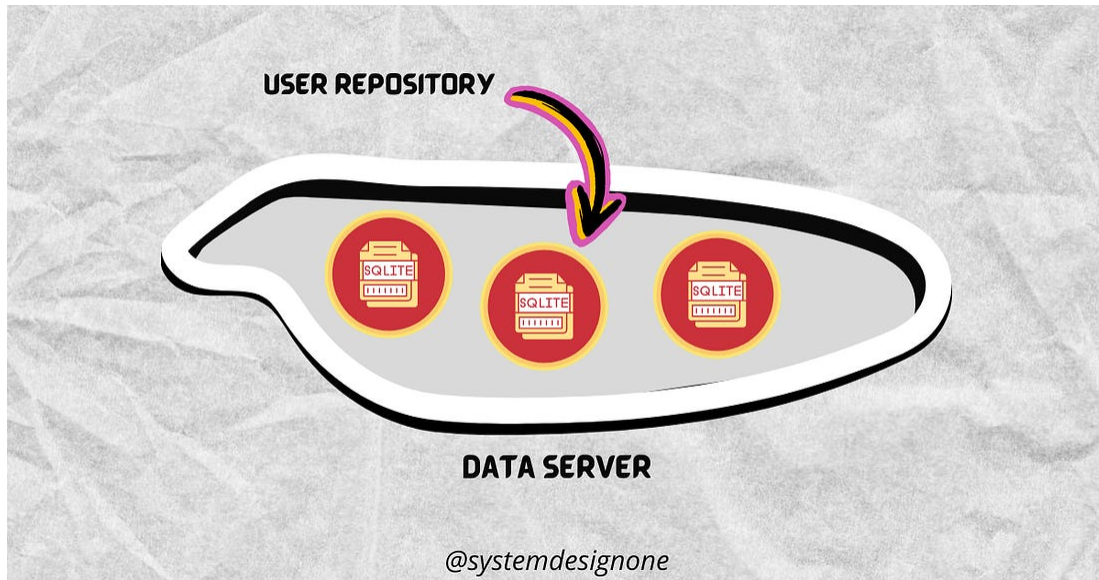
Here's how Bluesky works:

1. User Posts

A post is a short status update by a user; it includes text and images.

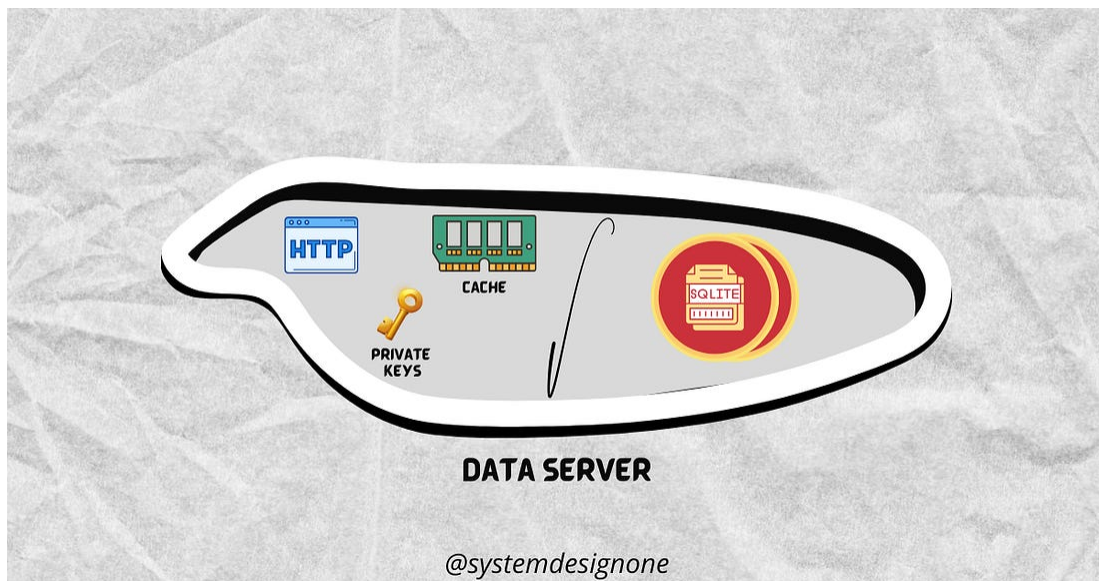
The text content and timestamp of the post are stored in a **repository**. *Think of the repository as a collection of data published by a single user.* SQLite is used as its data layer for simplicity; each repository gets a separate SQLite database.

The data records are encoded in CBOR, a compact binary format, before storing it in SQLite for low costs.



Storing User Data in Repositories

Repositories of different users are stored on a **data server**; they set up many data servers for scale.



Internals of Data Server

A data server exposes HTTP to handle client requests. Put simply, the data server acts as a proxy for all client interactions. Also it manages user

authentication and authorization. The data server includes tooling to automatically apply updates in the federated architecture.

They run 6 million user repositories on a single data server at 150 USD per month.

Think of the user repository as a Git repo and the data server as GitHub. It's easy to move a Git repo from GitHub to GitLab. Similarly, a user repository is movable from one data server to another.

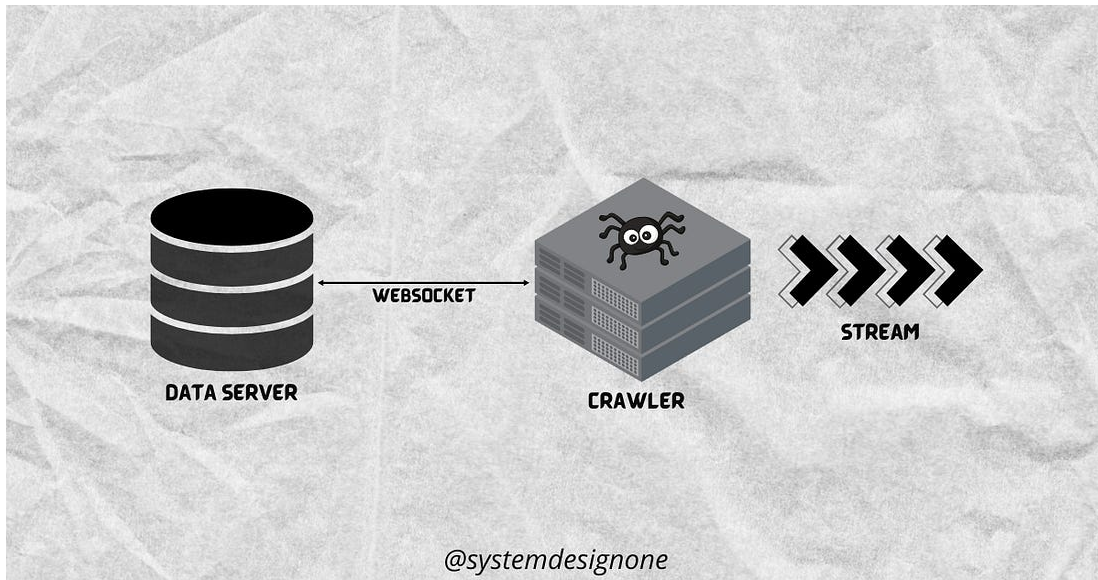
Besides it's possible to use alternative clients for Bluesky. Yet it's necessary to maintain a standard data schema for interactions. So separate **data schemas** and API endpoints are defined for each app on ATProto, including Bluesky.

A user's repository doesn't store information about actions performed by their followers such as comments or likes on their post. Instead, it's stored only in the repository of the follower who took the action.

A post is shown to the user's followers.

Yet it's expensive to push updates to each follower's repository. So information is collected from every data server using the **crawler**.

The crawler doesn't index data but forwards it.

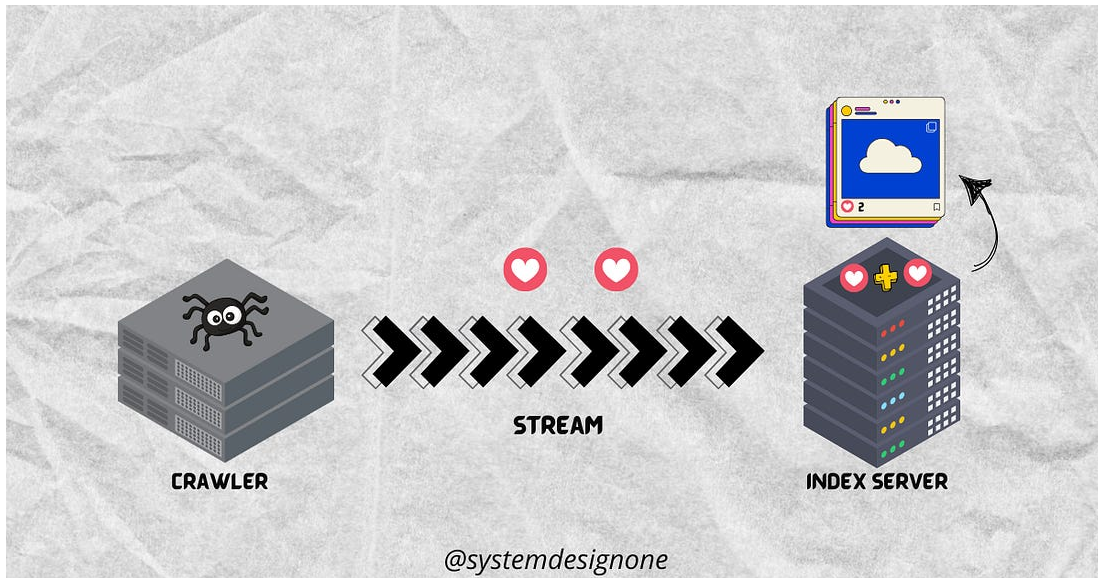


Crawler Collecting Information From Data Servers to
Generate the Stream

Here's how it works:

- The crawler subscribes for updates on the data server: new posts, likes, or comments.
- The data server notifies the crawler about updates in real time over websockets.
- The crawler collects information from data servers and generates a **stream**.

Consider the generated stream as a log over websockets; put simply, the crawler combines each user's actions into a single TCP connection.



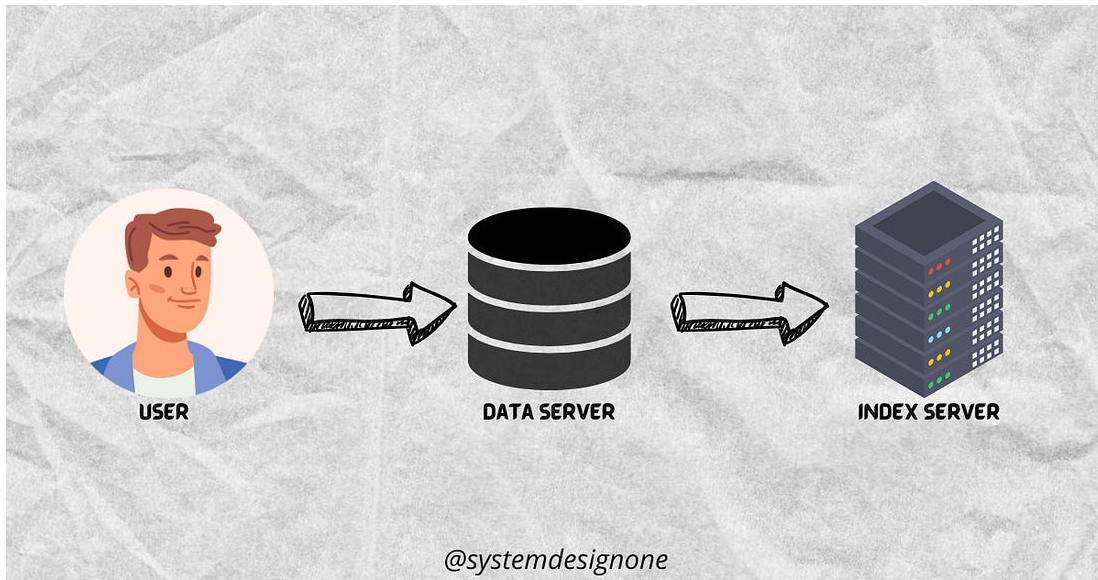
Aggregating a Post's Data From Stream Using the Index Server

A user's post is shown to followers only after counting the likes, comments, and reposts on it.

Yet the stream doesn't contain this information. So the stream's data is aggregated using the index server; it transforms raw data into a consumable form by processing it. Imagine the index server as a data presentation layer.

The index server is built using the Go language for concurrency. A NoSQL database, ScyllaDB, is used as its data layer for horizontal scalability.

A reference to the user's post ID is added to the follower's repository when they like or repost a post. So the total number of likes and reposts is calculated by crawling every user repository and adding the numbers.



Displaying a Post to the User

Here's the workflow for displaying a post:

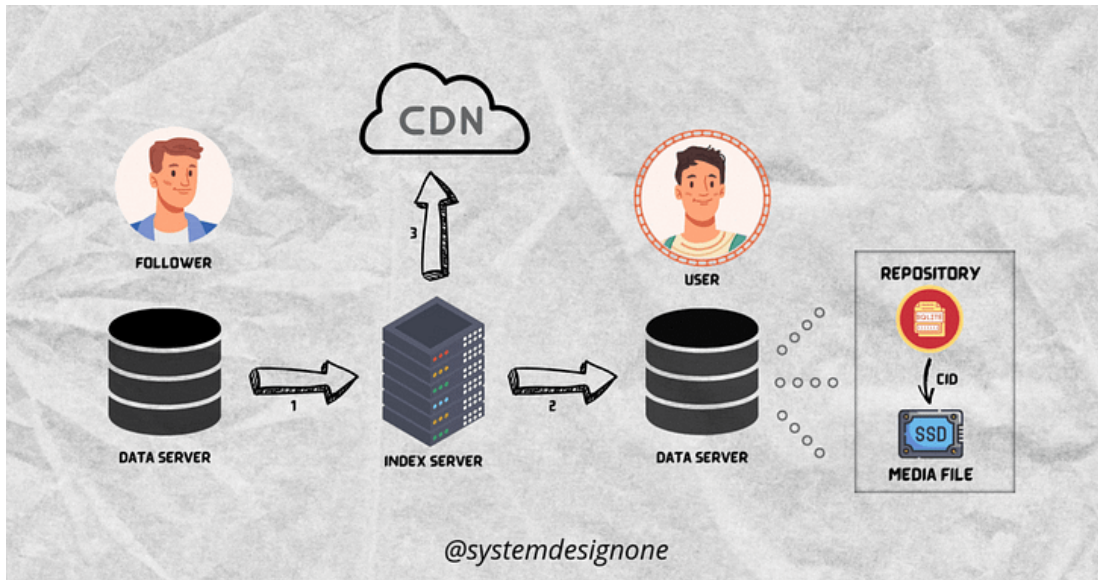
- A user's request is routed via their data server to the index server.
- The data server finds the people a user follows by looking at their repository.
- The index server creates a list of post IDs in reverse chronological order.
- The index server expands the list of post IDs to full posts with content.

The index server then responds to the client.

In short, a user repository stores primary data, while the index server stores derived data from repositories.

The index server is the most read-heavy service; so, its results are cached using Redis, an in-memory storage, for performance.

JSON Web Token (**JWT**) is used for authentication between Bluesky services.



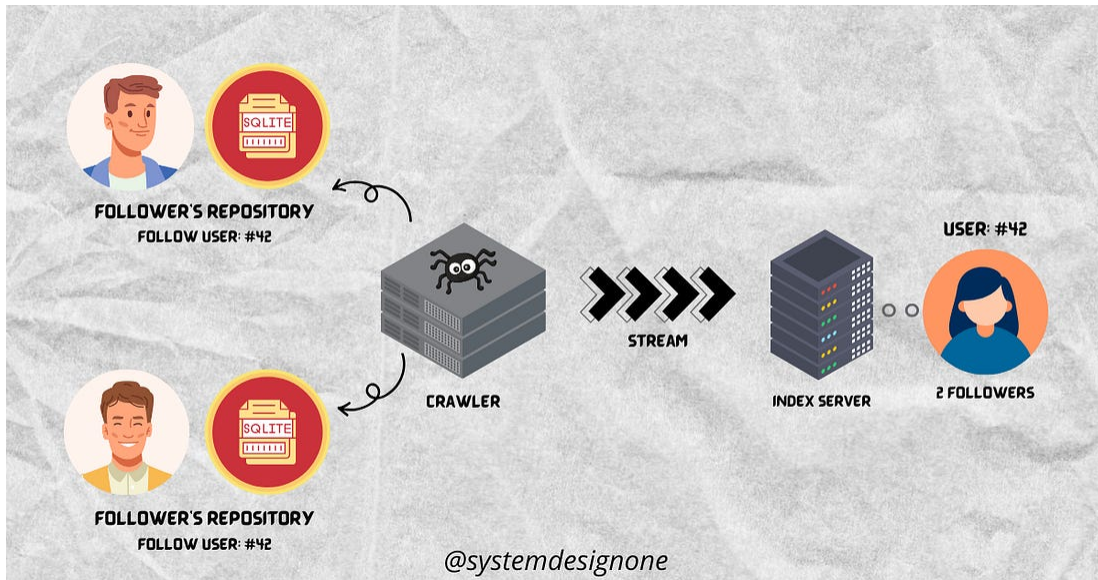
Serving Media Files via CDN for Efficiency

Media files, such as images and videos, are stored on the data server's disk for simplicity. A cryptographic ID (**CID**) is used to reference the media files in the repository. The index server fetches the media files from the data server on user request and caches them on the content delivery network (**CDN**) for efficiency.

2. User Profile

A user updates only their repository when they follow someone.

Their repository adds a reference to the user's unique decentralized identifier (**DID**) to indicate follow.



Counting the Number of Followers Using the Index Server

The number of followers for a user is found by indexing every repository. This is similar to how Google finds inbound links to a web page; all documents on the web are crawled.

A user account includes a handle based on the domain name (**DNS**); it keeps things simple.



Storing User Data Using Decentralized ID

And each user automatically gets a handle from the 'bsky.social' subdomain upon account creation. Yet posts are stored using DID, and the user handle is displayed along with the posts. So changes to a user's handle don't affect their previous posts.

A user's DID is immutable, but the handle is mutable; put simply, a user's handle is reassignable to a custom domain name.

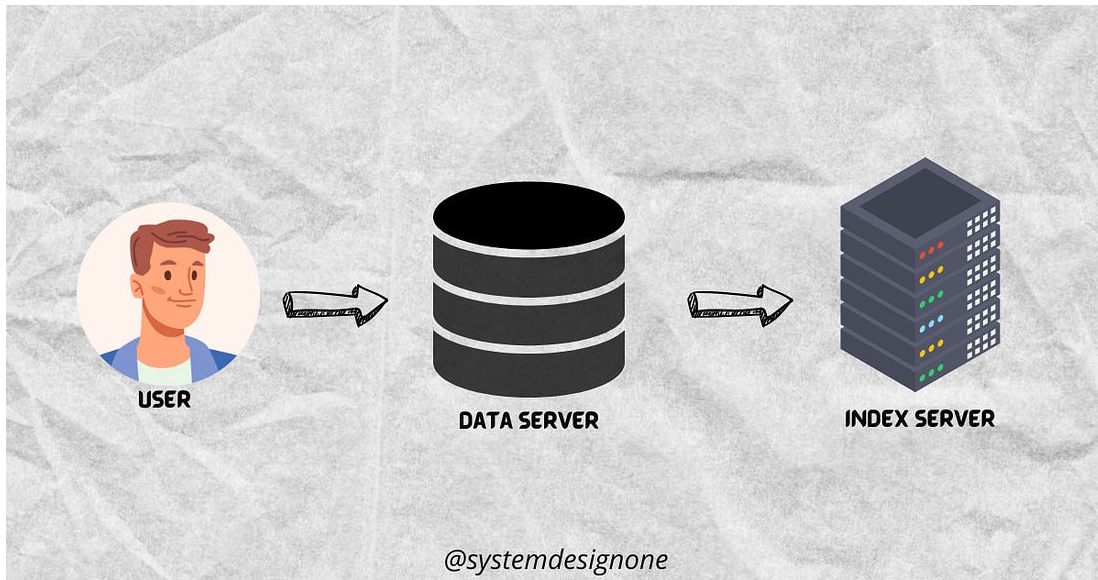
Here's how a user handle with a custom domain name is verified on Bluesky:

- The user enters their custom domain name in the Bluesky account settings.
- Bluesky generates a unique text value for the user: public key.
- The user stores this value in the DNS TXT record of the custom domain name.

The index server then periodically checks the DNS TXT record and validates the user handle. Imagine **DNS TXT record** as a text data field for domain name settings.

3. User Timeline and Feed

A user's timeline is created by arranging their posts in reverse chronological order.



Displaying a User's Timeline

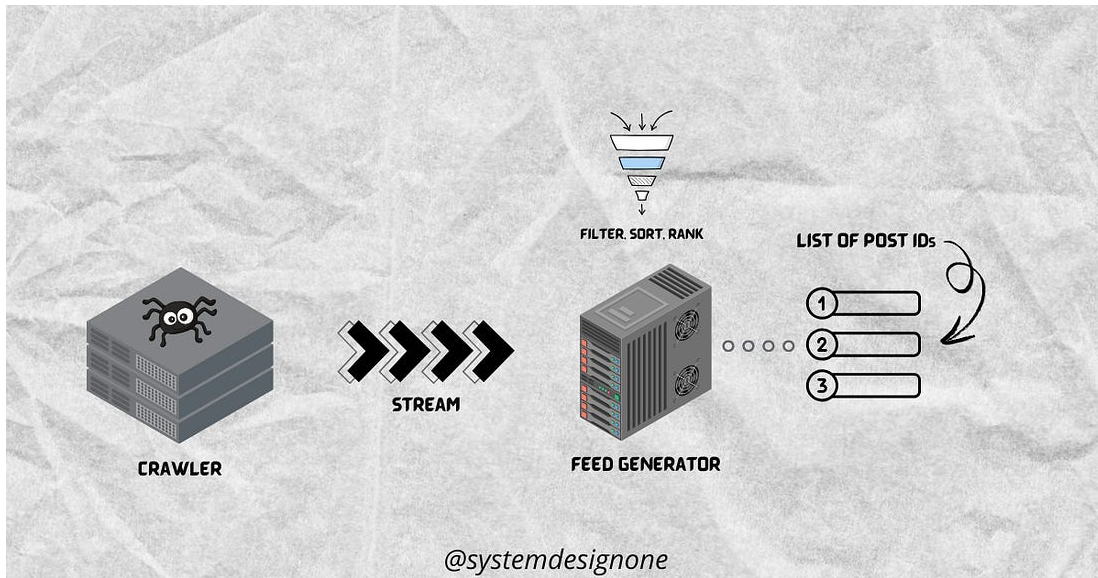
Yet a user's repository doesn't contain information about likes and comments received on a post. So the request is sent to the index server; it returns the user's timeline with aggregated data.

Besides the timeline of popular users is cached for performance.

A feed is created from posts by people a user follows.

Bluesky supports feeds with custom logic, and there are 50K+ custom feeds available.

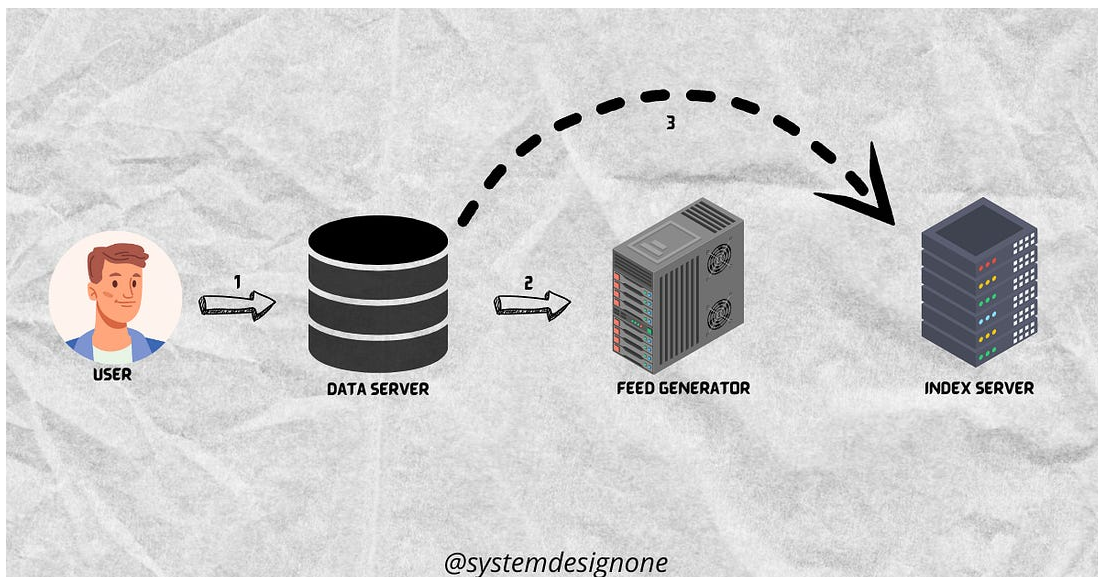
Consider the **custom feed** as a filter for a specific list of keywords or users.



Feed Generator Creating a Custom Feed

Here's how it works:

- The crawler generates a stream from data servers.
- The **feed generator**, a separate service, consumes the stream.
- The feed generator filters, sorts, and ranks the content based on custom logic.
- The feed generator creates a list of post IDs.

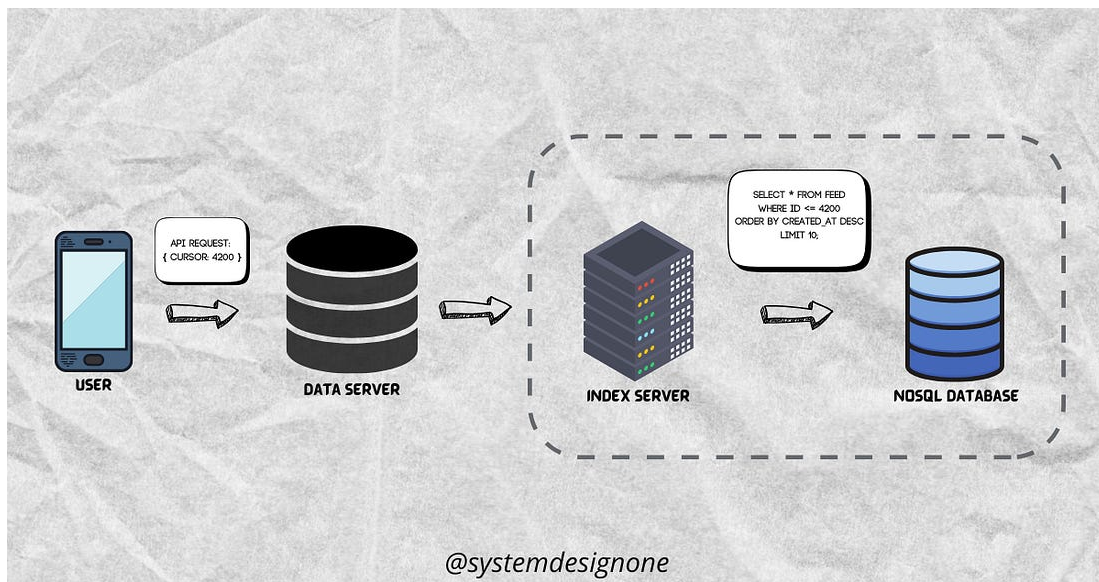


Serving a Custom Feed to the User

The index server then populates the feed's content on a user request.

Cursor-based pagination is used to fetch the feed; it offers better performance.

It includes an extra parameter in API requests and responses. The cursor parameter points to a specific item in the dataset; for example, the post's timestamp to fetch feed until a specific post.



Cursor-Based API Pagination for Performance

Here's how it works:

- A sequential unique column, such as the post's timestamp, is chosen for pagination.
- The user requests include the cursor parameter to indicate the result's offset.
- The index server uses the cursor parameter to paginate the result dataset.

- The index server responds with the cursor parameter for future requests.

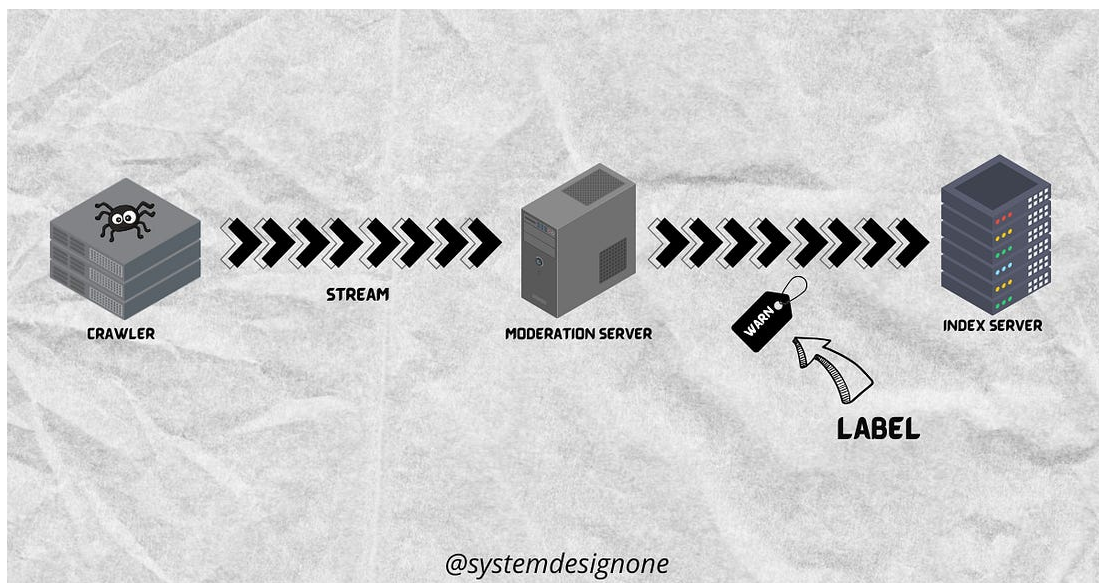
The client decides the cursor parameter's window size based on its viewport. The cursor-based pagination scales well as it prevents a full table scan in the index server.

4. Content Moderation

The content in Bluesky is analyzed using the **moderation service**; it assigns a label to the post.

Imagine a **label** as metadata to categorize content. Besides it's possible to apply a label manually. A user chooses to hide or show a warning for posts with a specific label.

This preference is stored on their data server.



Assigning Label to Content Using Moderation Service

Here's how Bluesky moderation works:

- The moderation service consumes the stream from the crawler.

- The moderation service analyzes the content and assigns a label to it.
- The index server stores the content along with its label.
- The user requests are routed via the data server to the index server.
- The data server includes label IDs in HTTP request headers before forwarding them.
- The index server applies the label setting in the response.

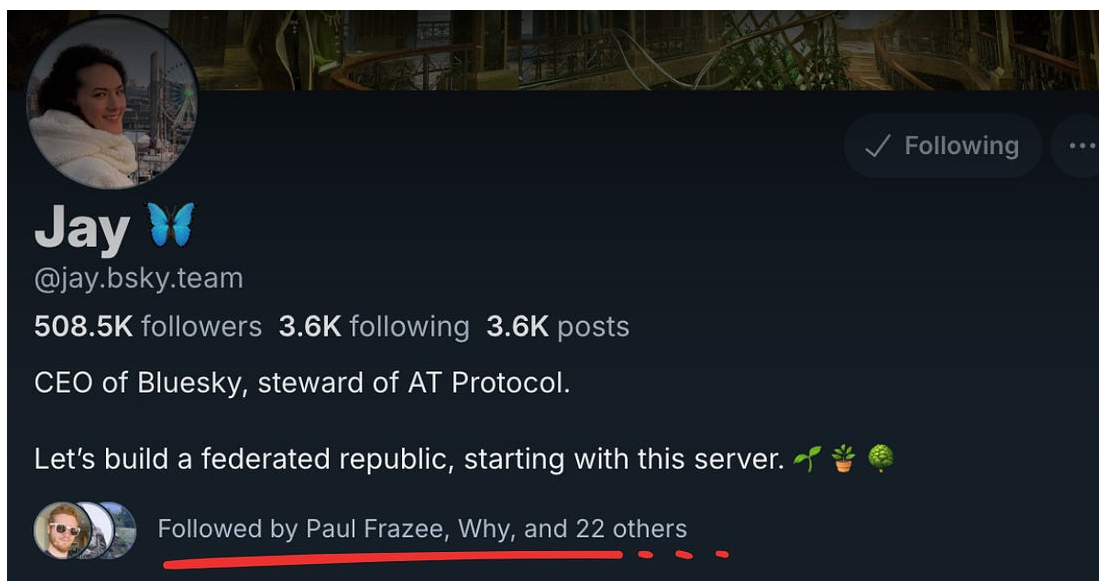
Also a data server operator does basic moderation.

The data server filters out the muted users before responding to the client.

Put simply, a user can post about anything, but the content is moderated before being shown to the public.

5. Social Proof

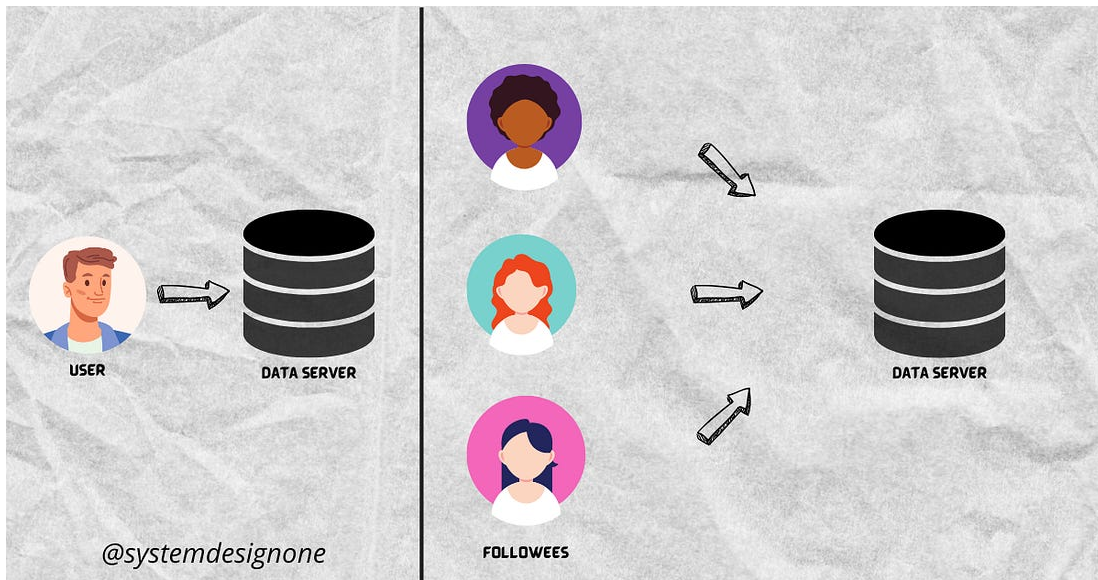
Bluesky allows a user to see how many people they follow also follow a specific user: social proof.



Social Proof Feature in Bluesky

For example, when I visit Jay's Bluesky profile, it shows how many people I follow also follow her.

Let's dive into the different approaches they used to build this feature.

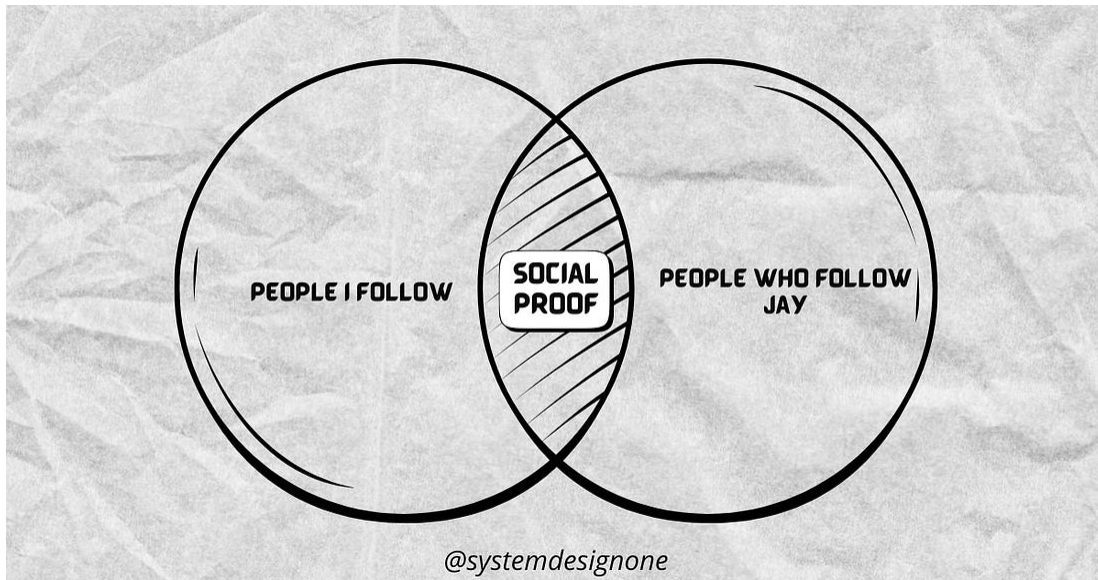


Computing Social Proof by Querying the Database

Here's a basic solution:

- Find the people I follow by querying the database.
- Do separate parallel queries for each user to find the people they follow.
- Check if any of them also follow Jay.

But this approach won't scale as the number of parallel queries increases with the number of people a user follows.

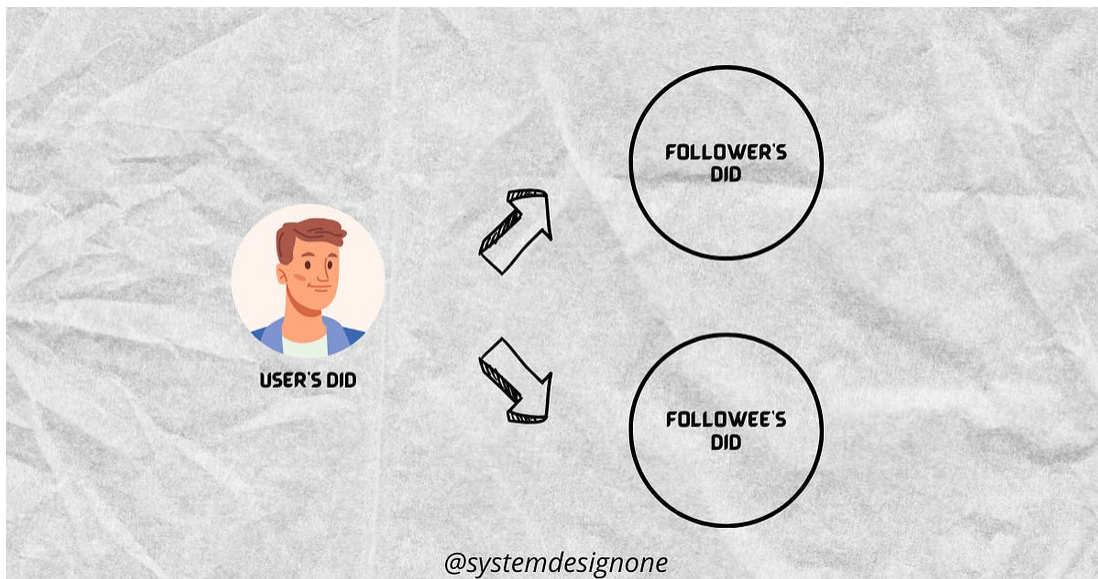


Social Proof Represented as a Set Intersection Problem

A scalable approach is to convert it into a set intersection problem:

- Set 1 tracks the people I follow.
- Set 2 tracks the people who follow Jay.

The intersection of these sets gives the expected result.



An In-Memory Graph Service to Track Follows

An in-memory graph service prevents expensive database queries and performs intersections quickly. Yet Redis Sets don't use different CPU cores at once. So here's how they implemented a minimum viable product:

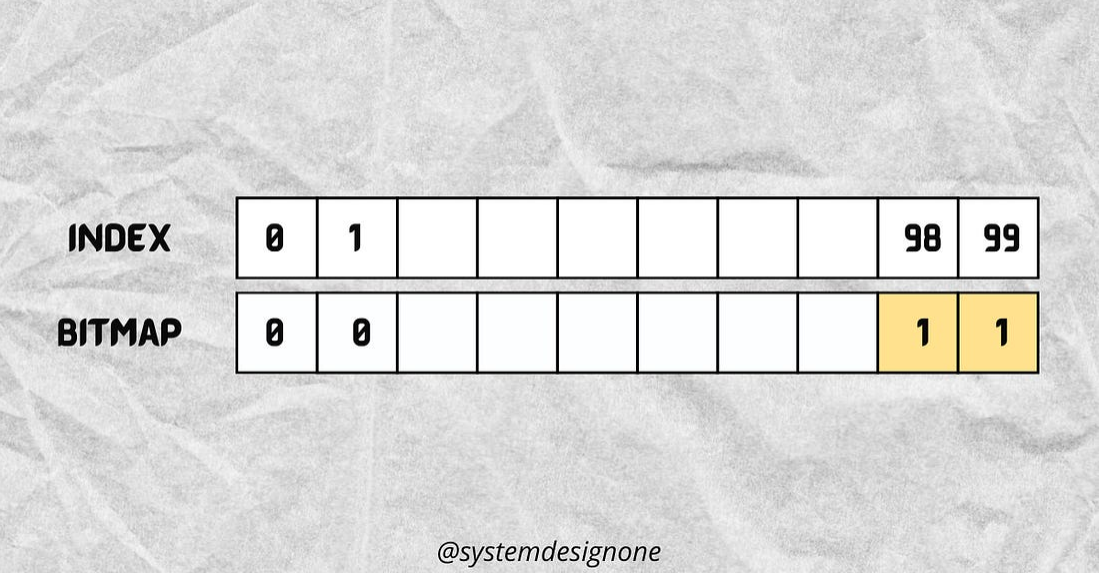
- Each user has a 32-character DID.
- The DID values are converted into uint64 to reduce memory usage.
- Each user's DID maintains 2 sets: people they follow and people who follow them.

But it still consumes a lot of memory and takes extra time to start.

So they optimized the graph service by implementing it using Roaring Bitmaps.

Yet let's take a step back and learn Bitmaps to better understand Roaring Bitmaps.

A **Bitmap** represents binary states using bit arrays.



INDEX	0	1						98	99
BITMAP	0	0						1	1

@systemdesignone

Tracking People I Follow Using Bitmap

Imagine Bluesky has only 100 users.

The index value of the people I follow is set to 1 on the Bitmap; the people I follow are then found by walking the Bitmap and recording the index of non-zero bits. A constant time lookup tells whether I follow a specific user. Although Bitmaps do faster bitwise operations, it's inefficient for sparse data.

A 100-bit long Bitmap is needed even if I follow only a single user; so, it won't scale for Bluesky's needs.

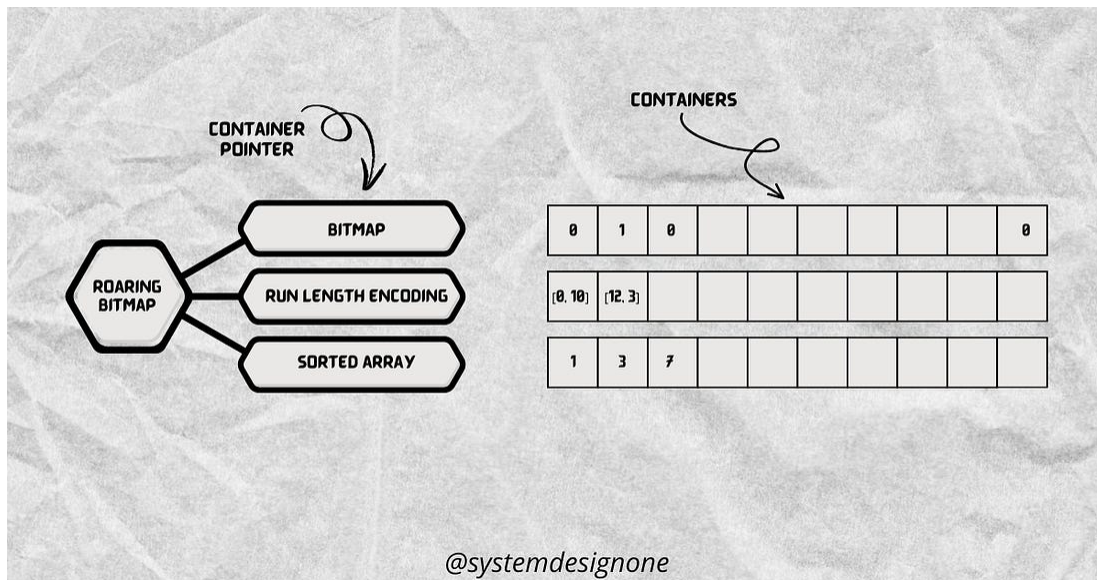
A better approach is to compress the Bitmap using **Run-Length Encoding**.

It turns consecutive 0s and 1s into a number for reduced storage. For example, if I follow the last 10 users, only the last 10 indices are marked by 1. With Run-Length Encoding, it's stored as 90 0s and 10 1s, thus low storage costs.

But this approach won't scale for randomly populated data, and a lookup walks the entire Bitset to find an index.

Ready for the best technique? **Roaring Bitmaps**.

Think of it as compressed Bitmaps, but 100 times faster. *A Roaring Bitmap splits data into containers and each container uses a different storage mechanism.*



Roaring Bitmaps to Track Social Proof

Here's how it works:

- The dense data is stored in a container using Bitmap; it uses a fixed-size bit array.
- The data with a large contiguous range of integers are stored in a container using run-length encoding; which reduces storage costs.
- The sparse data is stored in a container as integers in a sorted array; it reduces storage needs.

Put simply, Roaring Bitmaps use different containers based on data sparsity.

The set intersection is done in parallel and a container is converted into a different format for the set intersection. Also the graph data is stored and transferred over the network in Roaring Bitmap's serialization format.

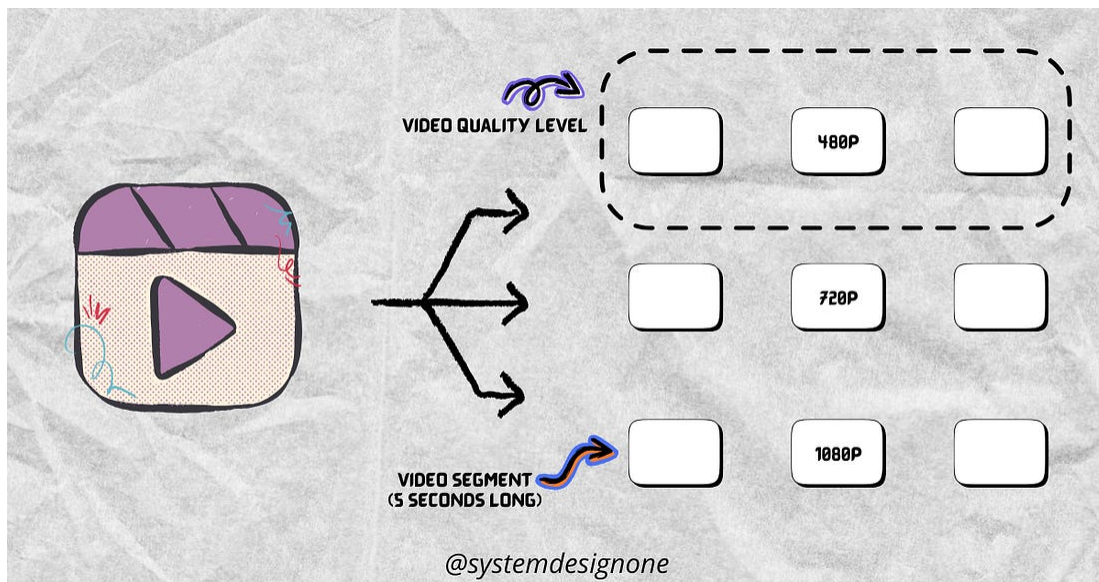
Many instances of the graph service are run for high availability and rolling updates.

6. Short Video

Bluesky supports short videos up to 90 seconds long.

The video is streamed through HTTP Live Streaming (**HLS**). Think of HLS as a group of text files; it's a standard for adaptive bitrate video streaming.

A client dynamically changes video quality based on network conditions.



How Adaptive Bitrate Video Streaming Works

Here's how it works:

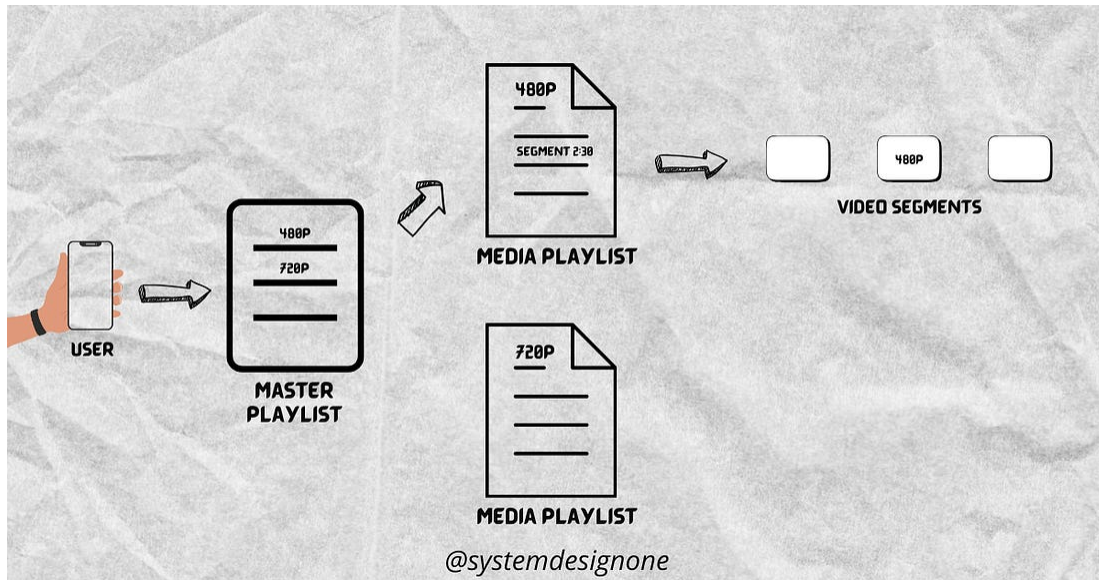
- A video is encoded into different quality levels: 480p, 720p, 1080p, and so on.
- Each encoded video is split into small segments of 5 seconds long.

The client checks the network conditions and requests video segments of the right quality.

HLS uses Playlist files to manage encoded video segments.

Imagine the **Playlist** as a text file, and there are 2 types of Playlists:

- Master playlist: list of all video quality levels available.
- Media playlist: list of all video segments for a specific video quality.



Selecting a Video Quality Level Based on Network Conditions

First, the client downloads the Master playlist to find available quality levels. Second, it selects a Media playlist based on network conditions. Third, it downloads the video segments in the sequence defined on the Media playlist.

The videos and playlists are cached in CDN to reduce costs and handle bandwidth needs. The stream requests are routed to CDN via [302 HTTP redirects](#).

The video views are tracked by [counting](#) requests to its Master playlist file.

And the response for the Master playlist includes a Session ID; it's included in future requests for Media playlists to track the user session.

Besides the video seconds watched by a user are found by checking the last fetched video segment.

A video subtitle is stored in text format and has a separate Media playlist file.

The Master playlist includes a reference to subtitles; here's how it works:

- The client finds the available subtitles by querying the Master playlist.
- The client downloads a specific subtitle's Media playlist based on the language selected by the user.

The client then downloads the subtitle segments defined in the Media Playlist.

Subscribe to get simplified system design case studies delivered to your inbox:

✓ **Subscribed**

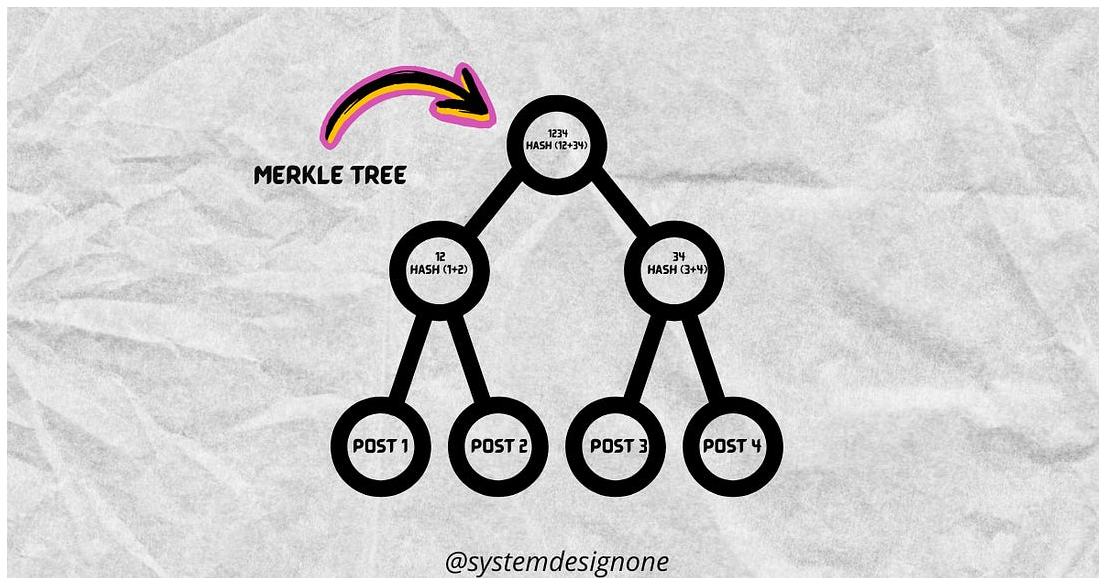
Bluesky Quality Attributes

Let's keep going:

1. Data Integrity

A user's data is cryptographically signed before saving it in the data server; it shows data authorship.

The Merkle search tree is used to check the integrity of data, such as posts, transferred over the network.



Checking Data Integrity Using Merkle Tree

Think of the **Merkle search tree** as an efficient data structure to check data integrity. It doesn't store original data; instead, it has only cryptographic hashes representing data.

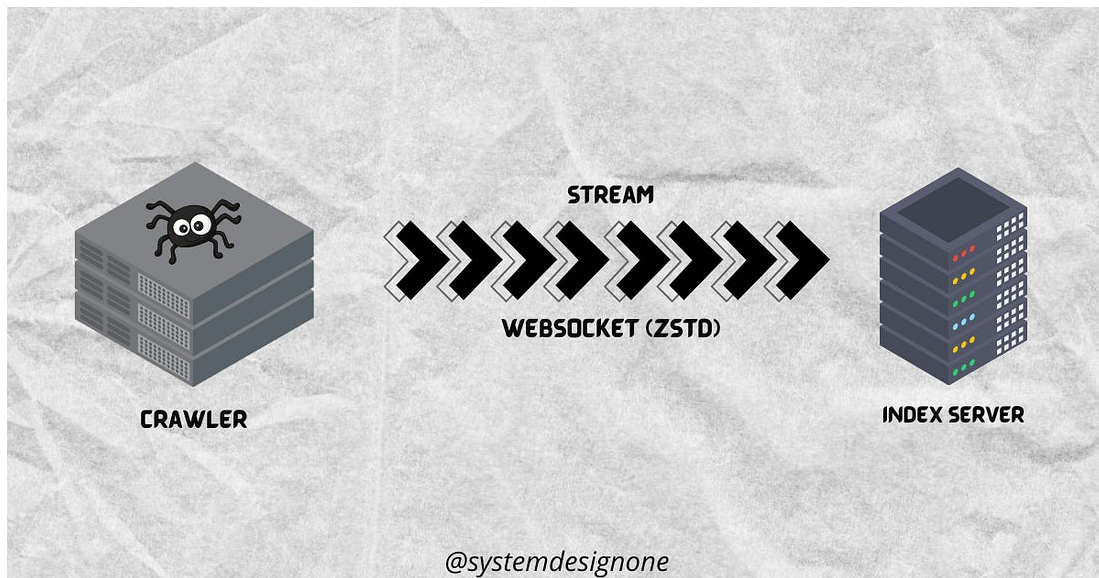
Besides the crawler keeps a copy of each user's repository; it verifies the Merkle search tree signatures on data updates.

2. Performance

The crawler generates a stream to update the index server.

Yet the stream increases bandwidth and infrastructure costs. So the events are validated on the crawler and services, such as the feed generator, are connected to a trusted crawler. Besides Merkle search tree blocks are converted to JSON objects, which lack verifiability and signatures, to reduce stream size.

Put simply, Merkle search tree verification overhead is avoided by consuming the stream from a trustable crawler.



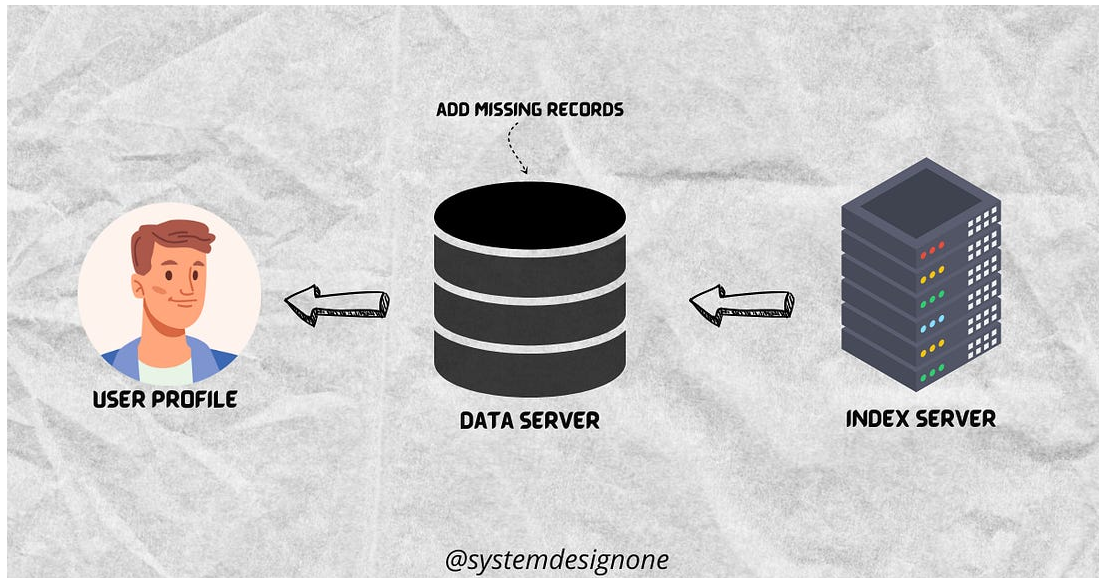
Compressing Websocket Traffic for Performance

The crawler transfers the stream to the index server over websockets; zstd compresses websocket traffic for performance.

A user with a massive following needs a data server with only tiny computing resources, as the index server displays aggregated data.

3. Consistency

Every change to a user's profile is propagated via the index server; which, means eventual consistency.



Read-After-Write Consistency for Profile Updates

Yet a user viewing their profile right after an update should see the latest changes. So Bluesky offers *read-after-write consistency for profile updates*; here's how:

- All client requests are routed via the data server to the index server.
- The response headers from the index server are compared against the data server.
- The response is updated to include missing records before returning it to the client.

This read-after-write behavior applies only to a user viewing their profile; others see the changes eventually.

4. Availability

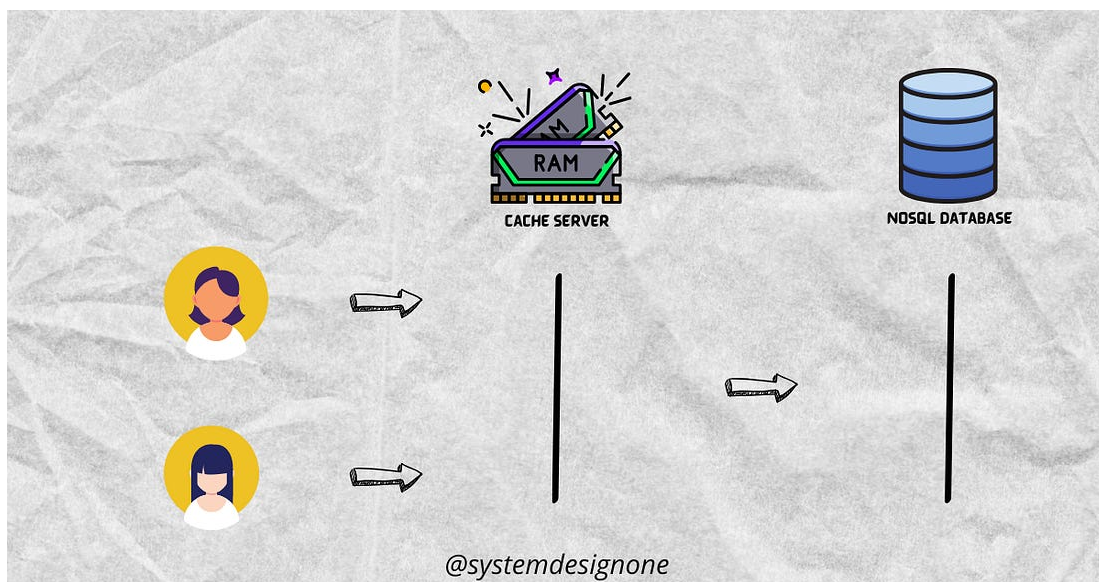
A popular user's post notifies many users, which results in many requests to view the post at once.

The results from the index server are cached to reduce the database load. Yet a popular user's post creates a thundering herd problem.

Here's how:

- A view request, the first one, from a follower reaches the cache server.
- The cache server queries the database for post content.
- Requests from others reach the database while the cache waits for the first response.

So request coalescing is done.



Request Coalescing to Reduce Database Load

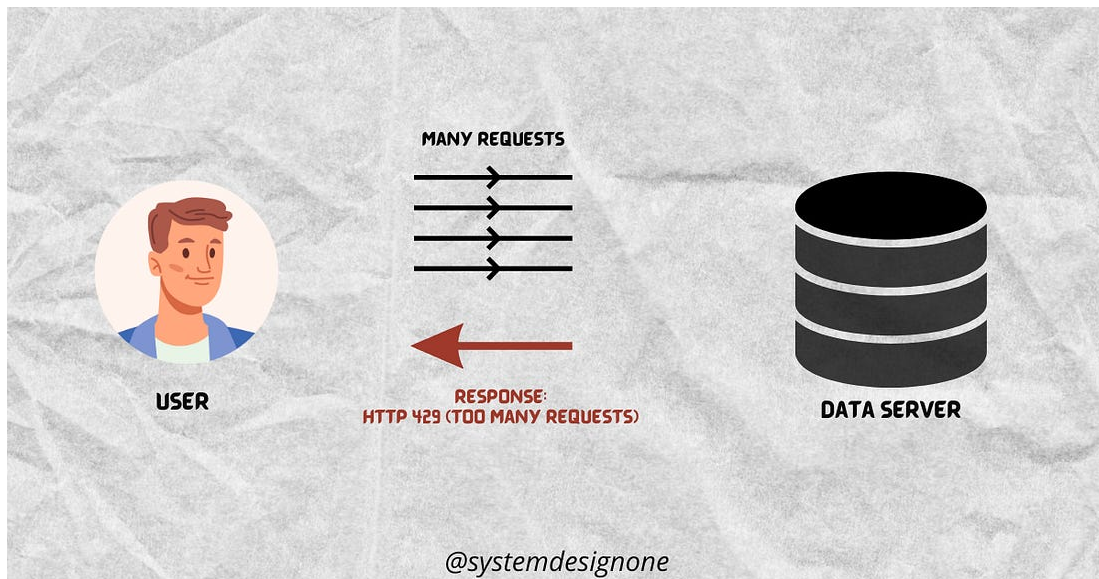
It groups requests for the same data within a short period. Then forwards only a single request to the database; thus, preventing the thundering herd problem.

5. Fault Tolerance

The data server notifies the crawler about data changes in repositories over websockets.

Yet network interruptions occur, so the crawler periodically checks repositories. It then compares the data against its local replica to find

data changes.

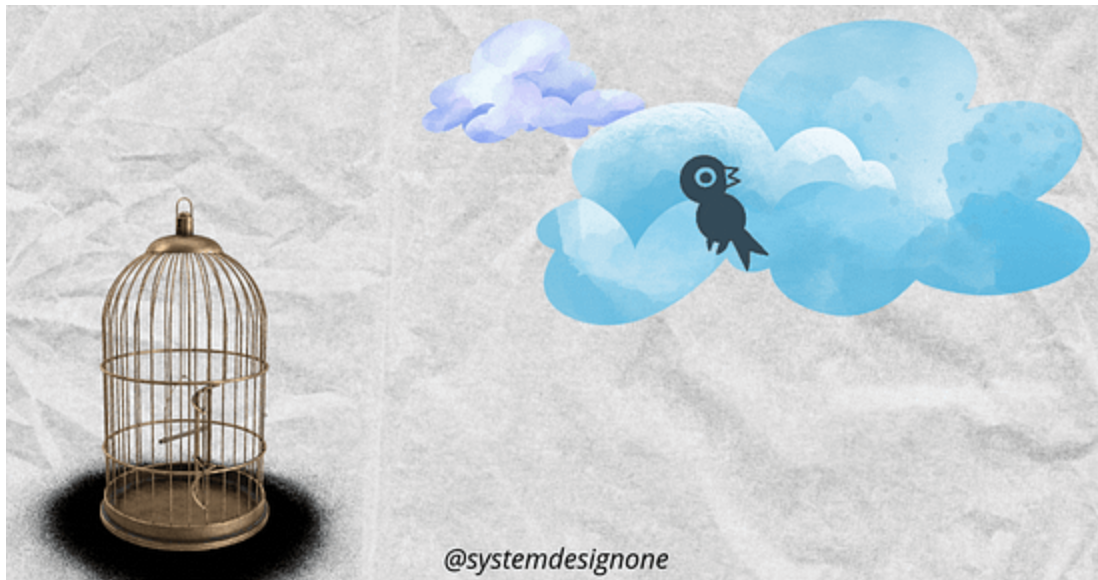


Rate Limiting Requests

Also requests are rate-limited to keep the system operational.

HTTP 429 (Too Many Requests) is returned in response headers when the rate limit threshold is exceeded. Interactions between servers are also rate-limited to avoid abuse in the decentralized architecture. A new data server's request limit capacity is increased over time, based on trust, to keep the crawler running smoothly.

Besides a user is only allowed up to 1,666 requests an hour to reduce bot activity.



Bluesky has more than 28 million users; it became an independent company in 2021.

Although anybody can host a server on ATProto, most Bluesky services are now run by a single company.

ATProto and Bluesky look like an interesting new approach to social networks.

Subscribe to get simplified case studies delivered straight to your inbox:

✓ Subscribed



Follow me on [Bluesky](#) | [LinkedIn](#) | [Threads](#) | [Twitter](#) |
[Instagram](#) | [YouTube](#)

Thank you for supporting this newsletter. Consider sharing this post with your friends and get rewards. Y'all are the best.

1 Referral



**Leetcode
Master
Template**

2 Referrals



**Interview
Mistakes
to Avoid PDF**

3 Referrals



**Popular
Interview
Questions PDF**

Share

References

- [Bluesky and the AT Protocol: Usable Decentralized Social Media](#)
- [Federation Architecture](#)
- [Bluesky and the AT Protocol - Presentation](#)
- [The AT Protocol](#)
- [Benefits of an open network](#)
- [Solving Thundering Herds with Request Coalescing in Go](#)
- [Jetstream: Shrinking the AT Proto Firehose by >99%](#)
- [Scaling Go to 192 Cores with Heavy I/O](#)
- [Your Data Fits in Memory \(GraphD Part 1\)](#)
- [An entire Social Network in 1.6GB \(GraphD Part 2\)](#)
- [Rate Limits](#)
- [How Does BlueSky Work?](#)
- [How HLS Works](#)
- [Bluesky and IPLD - Jay Graber](#)
- [Custom Feeds](#)
- [Labels and moderation](#)
- [Timestamps](#)
- [Service Auth](#)
- [Read-After-Write](#)
- [Bluesky's Moderation Architecture](#)
- [Bluesky on GitHub](#)
- [Bluesky experiments](#)
- [Decentralized Social Networks](#)
- [DID PLC Directory](#)
- [On Distributed Communications](#)
- [How to self-host all of Bluesky except the AppView](#)

- [A Self-Authenticating Social Protocol](#)
- [How decentralized is Bluesky really?](#)
- [Reply on Bluesky and Decentralization](#)
- [Re: Re: Bluesky and Decentralization](#)
- [Bluesky now has over 20M people](#)
- [Consistently faster and smaller compressed bitmaps with Roaring](#)
- [List of most-visited websites](#)